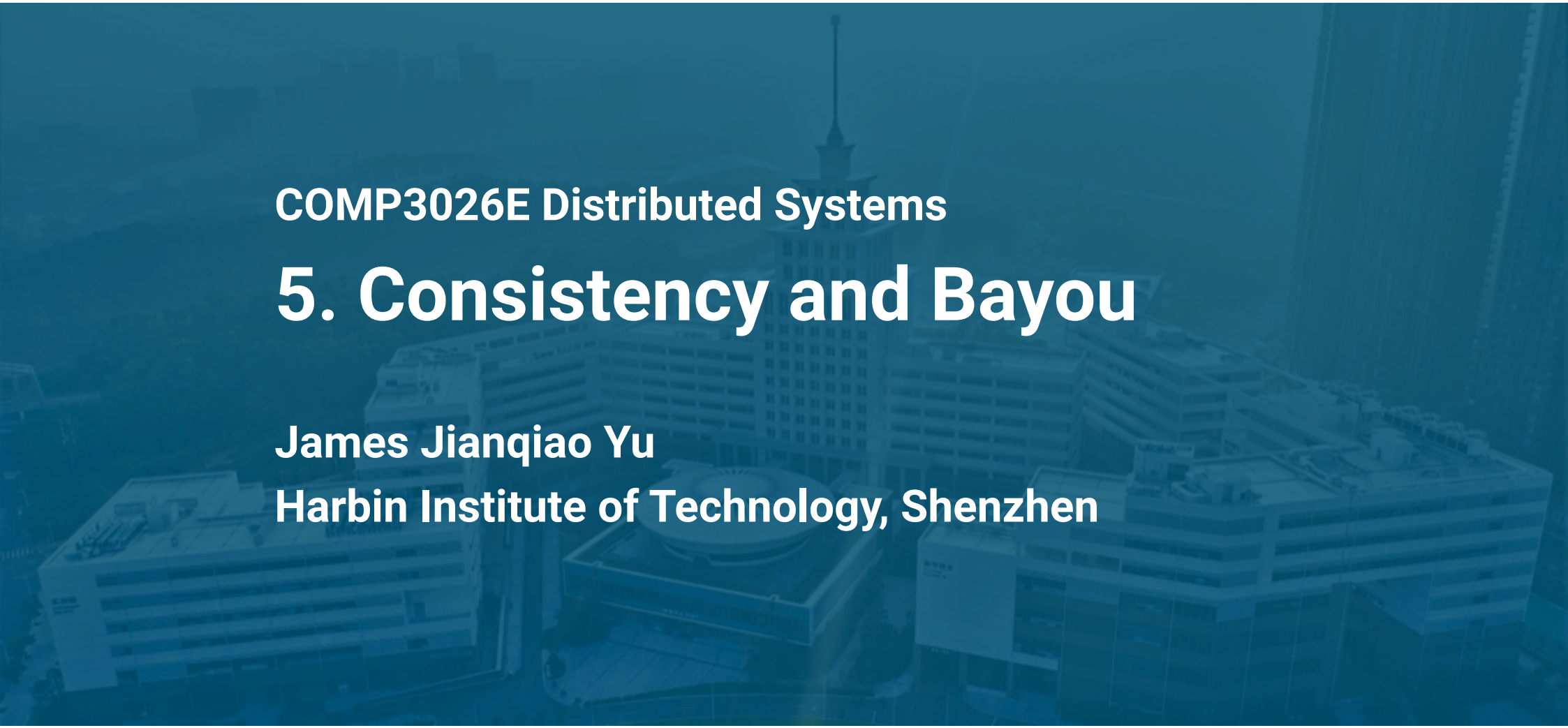


COMP3026E Distributed Systems

5. Consistency and Bayou

James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

Availability v.s. Consistency

- Later topic: Distributed consensus algorithms
  - Strong consistency (ops in same order everywhere)
  - But, strong reachability/availability requirements

If the network fails (common case), can we provide any consistency when we replicate?

Why Can't We Have Everything?

- In a **centralized** system, consistency is easy (one source of truth)
- In a **distributed** system, we want:
  1. **Availability**: Every request receives a (non-error) response
  2. **Partition** Tolerance: The system continues to operate despite network failures
  3. **Consistency**: All nodes see the same data at the same time

The Reality Networks are unreliable. Links break, packets drop, and latencies spike

The CAP Theorem (Brewer, 2000)

Theorem You can only pick TWO out of the three properties (C, A, P) at any given time

- The “P” is non-negotiable: In a distributed system, network partitions will happen
- So, the real choice is between **Consistency** and **Availability**

The CAP Theorem (Brewer, 2000)

Theorem You can only pick TWO out of the three properties (C, A, P) at any given time

- The “P” is non-negotiable: In a distributed system, network partitions will happen
- So, the real choice is between **Consistency** and **Availability**
  - CP (Consistency + Partition Tolerance): If the network breaks, stop responding to ensure data stays correct. (e.g., Banking)

The CAP Theorem (Brewer, 2000)

Theorem You can only pick TWO out of the three properties (C, A, P) at any given time

- The “P” is non-negotiable: In a distributed system, network partitions will happen
- So, the real choice is between **Consistency** and **Availability**
  - CP (Consistency + Partition Tolerance): If the network breaks, stop responding to ensure data stays correct. (e.g., Banking)
  - AP (Availability + Partition Tolerance): If the network breaks, keep responding even if the data is stale. (e.g., Social Media, Bayou)

The Reality of “Disconnected Operation”

- **Scenario**: You are a traveling researcher in 1995. You have a PDA (Personal Digital Assistant) and a laptop
- **Hardware**: No 5G, no ubiquitous Wi-Fi. Connectivity is via:
  - Occasional dial-up modems
  - Infrared (line-of-sight) beaming
  - Physical docking stations
- **Requirement**: You must be able to schedule meetings or edit documents while on a plane or in a basement, with zero network access
- **Conflict**: What happens when two people, both offline, edit the same file?

Eventual Consistency

- **Eventual consistency**: If no new updates to the object, **eventually** all reads will return the last updated value
- Common: git, iPhone sync, OneDrive, etc.

Eventual Consistency

- **Eventual consistency**: If no new updates to the object, **eventually** all reads will return the last updated value
- Common: git, iPhone sync, OneDrive, etc.
- Why do people like eventual consistency?
  - Fast read/write of local copy of data
  - Disconnected operation

Eventual Consistency

- **Eventual consistency**: If no new updates to the object, **eventually** all reads will return the last updated value
- Common: git, iPhone sync, OneDrive, etc.
- Why do people like eventual consistency?
  - Fast read/write of local copy of data
  - Disconnected operation

Issue Conflicting writes to different copies. How to reconcile them when discovered?

An Example

- Meeting room calendar application as case study in ordering and conflicts in a distributed system with poor connectivity
- Each calendar entry = room, time, set of participants
- Want everyone to see the same set of entries, **eventually**
  - Else users may double-book room
    - or avoid using an empty room

Challenges of Eventual Consistency

- **"Eventually"** Problem: How long is "eventually"? Minutes? Days?
- **Conflict** Problem: If two people edit the same calendar slot while offline, who wins?
- **Ordering** Problem: If I delete a meeting and then add a new one, but you receive the "Add" before the "Delete," what happens?
- **User** Problem: How do we prevent the user from seeing "time-traveling" data (data that disappears and reappears)?

Why Not Just a Central Server?

- Want my calendar on a **disconnected** mobile phone
  - i.e., each user wants database replicated on their device
  - Not just a single copy
- Want **ad-hoc** connectivity
  - e.g., Alice and Bob see each other at coffee shop and synchronize databases

Bayou: Weakly Connected Replicated Storage

- Early ‘90s: Dawn of PDAs, laptops
  - H/W clunky but showing clear potential
  - Commercial devices did not have wireless
- **Bayou**: A system designed specifically to handle these “Partitioned” (P) and “Available” (A) scenarios
  - Support data sharing among users only occasionally connected

Managing Update Conflicts in Bayou,
a Weakly Connected Replicated Storage System

Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers,
Mike J. Spreitzer and Carl H. Hauser

Computer Science Laboratory
Xerox Palo Alto Research Center
Palo Alto, California 94304 U.S.A.

The Bayou Approach

- **Application-Specific**: The system shouldn't try to solve conflicts globally. Let the applications decide how to fix a calendar conflict
- **The Log is the Truth**: The database "state" (the grid you see) is just a projection. The Log of Writes is the permanent record
- **Dependency Tracking**: Every write should "know" what state it expects to find before it executes

How to Synchronize Databases?

- Suppose two users are in Bluetooth range
  - Each sends entire calendar database to other
  - Possibly expend lots of network bandwidth

How to Synchronize Databases?

- Suppose two users are in Bluetooth range
  - Each sends entire calendar database to other
  - Possibly expend lots of network bandwidth
- What if the calendars **conflict**, e.g., the two calendars have concurrent meetings in a room?
  - iPhone sync keeps both meetings

How to Synchronize Databases?

- Suppose two users are in Bluetooth range
  - Each sends entire calendar database to other
  - Possibly expend lots of network bandwidth
- What if the calendars **conflict**, e.g., the two calendars have concurrent meetings in a room?
  - iPhone sync keeps both meetings
  - Want to do better: **automatic conflict resolution**

Automatic Conflict Resolution

- Can't just view the calendar database as abstract bits:
  - Too little information to resolve conflicts:

1. "Both files have changed" can **falsely conclude** calendar conflict
   - e.g., Monday 10am meeting in room 3 and Tuesday 11am meeting in room 4
2. "Distinct record in each database changed" can **falsely conclude** no conflict
   - e.g., Monday 10–11am meeting in room 3 Doug attending, and Monday 10–11am meeting in room 4 Doug attending, …

Application-Specific Conflict Resolution

- Intelligence that can identify and resolve conflicts
  - More like users' updates: read database, think, change request to eliminate conflict
  - Must ensure all nodes resolve conflicts in the same way to keep replicas consistent

Application-Specific Update Functions

- Suppose calendar write takes form:
  - “10am meeting, Room=302, COMP3026E staff”
  - How would this handle conflicts?

Application-Specific Update Functions

- Suppose calendar write takes form:
  - “10am meeting, Room=302, COMP3026E staff”
  - How would this handle conflicts?
- Better: write is an update function for the app
  - “1-hour meeting at 10am if room is free, else 11am, Room=302, COMP3026E staff”

What is in a “Bayou Write”?

- In Bayou, a write is a **bundle of logic** sent by the client

Problem By the time my “Write” reaches your server, the data it was based on might have changed.

- Bayou Solution: Every write operation includes:
  - The Proposed **Update**: The data the user wants to change.
  - A **Dependency** Check: A precondition that must be true for the update to work.
  - A **Merge** Procedure: A “Plan B” if the precondition fails.

“1-hour meeting at 10am if room is free, else 11am, Room=302, COMP3026E staff”

Mechanism 1: Dependency Checks

Purpose To detect a conflict without requiring a human to look at it.

- How it works: The application provides a query and an expected result
- Example:
  - **Update**: Book Room 302 at 10:00 AM
  - **Dependency** Check: “Is Room 302 currently empty at 10:00 AM?”
- Execution: Before applying the update, the replica runs the check
  - If `Check == True`: Apply the update
  - If `Check == False`: A conflict is detected! Trigger the **Merge** Procedure

Mechanism 2: Application-Specific Merge

Purpose To resolve conflicts automatically based on “Business Logic”

- Concept: Instead of failing or asking the user, the app provides a script to fix the issue
- Example:
  - **Logic**: “If Room 302 is busy, try to book Room 303. If 303 is busy, try 11:00 AM. If all else fails, log a ‘Failed to Book’ entry in the user’s notifications”
- Key **Benefit**: The system remains autonomous. Replicas can resolve conflicts even while the user is asleep or offline

The Cost of Programmable Writes

- **Complexity** for Developers: Developers can't just write INSERT; they have to write conflict-resolution scripts for every possible scenario
- **Determinism** is Mandatory: The Merge Procedure must be deterministic
  - If Node A and Node B run the same Merge Procedure on the same data, they must get the exact same result
    - You cannot use `gettime()` or `random()` inside a Bayou Merge Procedure
- **Interpretation**: Replicas must run an interpreter (like a mini-virtual machine) to execute these scripts safely

Problem: Permanently Inconsistent Replicas

- Node A asks for meeting M1 at 10am, else 11am
- Node B asks for meeting M2 at 10am, else 11am

- Node X syncs with A, then B
- Node Y syncs with B, then A

Problem: Permanently Inconsistent Replicas

- Node A asks for meeting M1 at 10am, else 11am
- Node B asks for meeting M2 at 10am, else 11am

- Node X syncs with A, then B
- Node Y syncs with B, then A

- X will put meeting M1 at **10:00**
- Y will put meeting M1 at **11:00**

Can’t just apply update functions when replicas sync

Totally Order the Updates!

- Maintain an ordered list of updates at each node
  - Make sure every node holds same updates
    - And applies updates in the same order
  - Make sure updates are a deterministic function of database contents
- If we obey above, “sync” is simple merge of two ordered lists
  - Each node applies updates in same order, so replicas remain consistent

Agreeing on the Update Order

- Timestamp: ⟨`local timestamp T, originating node ID`⟩
- Ordering updates `a` and `b`:
  - `a` < `b` if `a.T` < `b.T` or (`a.T` = `b.T` and `a.ID` < `b.ID`)

Write Log Example

- `〈701, A〉`: A asks for meeting M1 at 10am, else 11am
- `〈770, B〉`: B asks for meeting M2 at 10am, else 11am

- Pre-sync database state:
  - A has M1 at 10am
  - B has M2 at 10am

- What's the **correct eventual outcome**?
  - The result of executing update functions in timestamp order: M1 at 10am, M2 at 11am

Write Log Example: Sync Problem

- `〈701, A〉`: A asks for meeting M1 at 10am, else 11am
- `〈770, B〉`: B asks for meeting M2 at 10am, else 11am

- Now A and B sync with each other. Then:
  - Each sorts new entries into its own log
    - Ordering by timestamp
  - Both now know the full set of updates

Write Log Example: Sync Problem

- `〈701, A〉`: A asks for meeting M1 at 10am, else 11am
- `〈770, B〉`: B asks for meeting M2 at 10am, else 11am

- Now A and B sync with each other. Then:
  - Each sorts new entries into its own log
    - Ordering by timestamp
  - Both now know the full set of updates
- A can just run B's update function

Write Log Example: Sync Problem

- `〈701, A〉`: A asks for meeting M1 at 10am, else 11am
- `〈770, B〉`: B asks for meeting M2 at 10am, else 11am

- Now A and B sync with each other. Then:
  - Each sorts new entries into its own log
    - Ordering by timestamp
  - Both now know the full set of updates
- A can just run B's update function
- But B has **already** run B's operation, **too soon!**

Solution: Roll Back and Replay

- B needs to **“roll back”** the database, and re-run both ops in the correct order
- Bayou User Interface: Displayed meeting room calendar entries are **“Tentative” at first**
  - B’s user saw M2 at 10am, then it moved to 11am

Big Point The log at each node holds the truth; the database is just an optimization

Database vs. The Log

- The “Database” (the current calendar view) is just a temporary cache
- **Write Log** is the only “Source of Truth”
- The Process:
  1. A new write arrives.
  2. It is added to the log in a specific order (using Lamport Clocks).
  3. The replica “rolls back” the database to a known state.
  4. The replica “replays” the log from that point, running every Dependency Check and Merge Procedure in order.

Result Every replica that has the same log will eventually reach the same database state

Managing User Expectations

- Because logs are replayed and re-ordered, the state of your calendar might change!
  - 9:00 AM: You book a room (It looks successful)
  - 9:05 AM: You sync with a colleague who booked that room at 8:55 AM
  - 9:06 AM: Your booking is “pushed” later in the log. Your Merge Procedure moves your meeting to 11:00 AM

Managing User Expectations

- Because logs are replayed and re-ordered, the state of your calendar might change!
  - 9:00 AM: You book a room (It looks successful)
  - 9:05 AM: You sync with a colleague who booked that room at 8:55 AM
  - 9:06 AM: Your booking is “pushed” later in the log. Your Merge Procedure moves your meeting to 11:00 AM
- The UI Challenge: How do we tell the user that their data is “subject to change”?
- Bayou Solution: Label data as Tentative (not yet finalized) or Committed (stable)

Does Update Order Respect Causality?

- ⟨701, A⟩ : A asks for meeting M1 at 10am, else 11am
- ⟨700, B⟩ : Delete update ⟨701, A⟩
  - Possible if B's clock is slow, and using real-time timestamps

Does Update Order Respect Causality?

- ⟨701, A⟩: A asks for meeting M1 at 10am, else 11am
- ⟨700, B⟩: Delete update ⟨701, A⟩
  - Possible if B's clock is slow, and using real-time timestamps
- Result: delete will be ordered before add
  - (Delete never has an effect.)

Does Update Order Respect Causality?

- `〈701, A〉`: A asks for meeting M1 at 10am, else 11am
- `〈700, B〉`: Delete update `〈701, A〉`
  - Possible if B's clock is slow, and using real-time timestamps
- Result: delete will be ordered before add
  - (Delete never has an effect.)
- Q: How can we assign timestamp to respect causality?

Lamport Clocks Respect Causality

- Want event timestamps so that if a node observes E1 then generates E2, then `TS(E1)` < `TS(E2)`
- Use Lamport clocks!
  - If E1 $\rightarrow$ E2 then `TS(E1)` < `TS(E2)`

Lamport Clocks Respect Causality

- `⟨701, A⟩` : A asks for meeting M1 at 10am, else 11am
- `⟨700, B⟩` : Delete update `⟨701, A⟩`

Lamport Clocks Respect Causality

- `⟨701, A⟩` : A asks for meeting M1 at 10am, else 11am
- **`⟨706, B⟩`** : Delete update `⟨701, A⟩`

- With Lamport clocks:
  - When A sends `⟨701, A⟩`, it includes its clock, T (> 701)
  - When B receives `⟨701, A⟩`, it updates its clock to $T' > T$
  - When B creates the delete, it timestamps it with its clock, $T'' > T'$
  - $T'' > T' > T > 701$ (e.g., T'' is 706)

Lamport Clocks Respect Causality

- `〈701, A〉`: A asks for meeting M1 at 10am, else 11am
- **`〈706, B〉`**: Delete update `〈701, A〉`

- With Lamport clocks:
  - When A sends `〈701, A〉`, it includes its clock, T (> 701)
  - When B receives `〈701, A〉`, it updates its clock to $T' > T$
  - When B creates the delete, it timestamps it with its clock, $T'' > T'$
  - $T'' > T' > T > 701$ (e.g., T'' is 706)

Question What if A and B are concurrent?

Timestamps for Write Ordering: Limitations

- Never know whether some write from “the past” may yet reach your node…

Timestamps for Write Ordering: Limitations

- Never know whether some write from “the past” may yet reach your node…
  - So all entries in log must be **tentative forever**
  - And you must **store entire log forever**

Want to **commit** a tentative entry, so we can trim logs and have meetings

Fully Decentralized Commit

- Strawman: Update `⟨10, A⟩` committed when all nodes have seen all updates with TS ≤ 10
- Have sync always send in log order
- If you have seen updates with TS > 10 from every node then you'll never again see one < `⟨10, A⟩`
  - So `⟨10, A⟩` is committed

Fully Decentralized Commit

- Strawman: Update ⟨10, A⟩ committed when all nodes have seen all updates with TS ≤ 10
- Have sync always send in log order
- If you have seen updates with TS > 10 from every node then you'll never again see one < ⟨10, A⟩
  - So ⟨10, A⟩ is committed
- Why doesn't Bayou do this?
  - A node that remains disconnected prevents commiting
    - So many writes may be rolled back on re-connect

How Bayou Commits Writes

- Bayou uses a primary commit scheme
  - One designated node (the primary) commits updates

How Bayou Commits Writes

- Bayou uses a primary commit scheme
  - One designated node (the primary) commits updates
- Primary marks each write it receives with a permanent CSN (commit sequence number)
  - That write is committed
  - Complete timestamp = `⟨CSN, local TS, node-id⟩`

Advantage Can pick a primary node close to locus of update activity

How Bayou Commits Writes

- Nodes exchange CSNs when they sync
- CSNs define a total order for committed writes
  - All nodes eventually agree on the total order
  - Tentative writes come after all committed writes

Committed vs. Tentative Writes

- If a node has a write with a CSN:
  - It has seen all CSNs ≤ that write by the propagation protocol (exchange full logs)
  - Can then show user the write has **committed**
    - Mark calendar entry “Confirmed” instead of “Tentative”

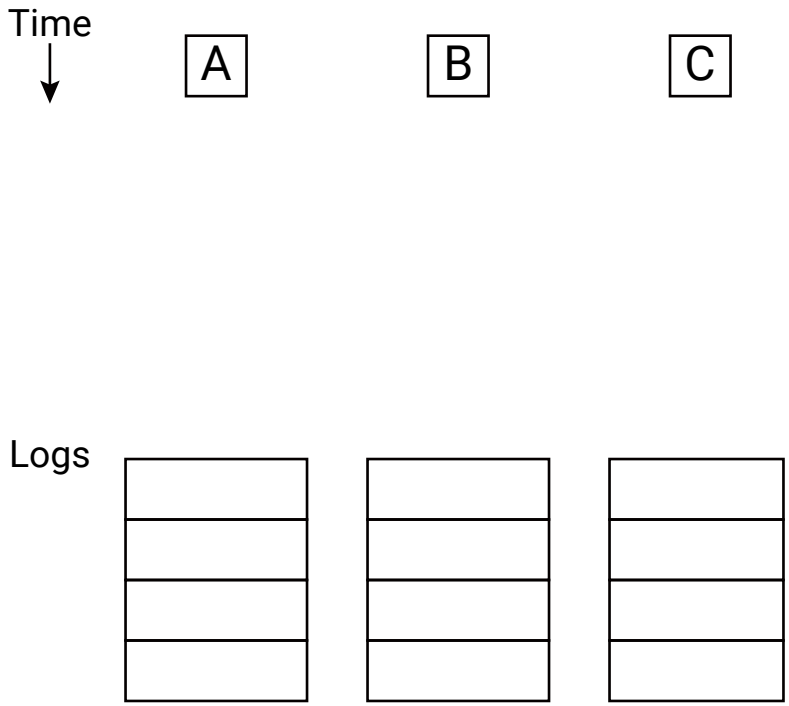
Committed vs. Tentative Writes

- If a node has a write with a CSN:
  - It has seen all CSNs ≤ that write by the propagation protocol (exchange full logs)
  - Can then show user the write has **committed**
    - Mark calendar entry “Confirmed” instead of “Tentative”
- Slow/disconnected node cannot prevent commits!
  - Primary replica allocates CSNs

Tentative Writes

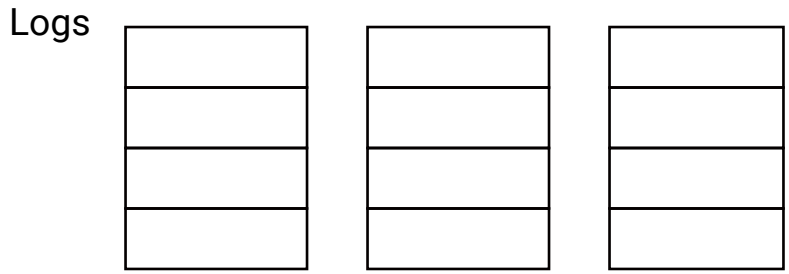
- What about tentative writes, though? How do they behave, as seen by users?
- Two nodes may disagree on meaning of tentative writes
  - Even if those two nodes have synced with each other!
  - Only CSNs from primary replica can resolve disagreements permanently

Example: Disagreement on Tentative Writes



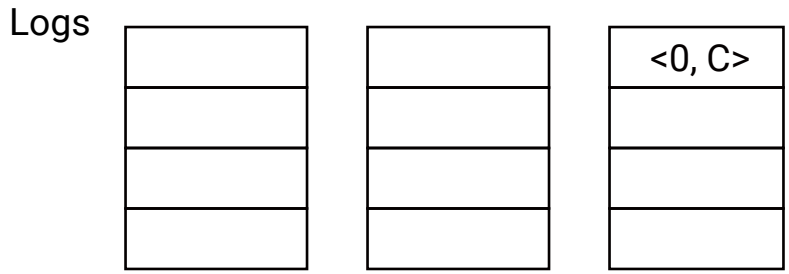
- ⟨local TS, node-id⟩

Example: Disagreement on Tentative Writes



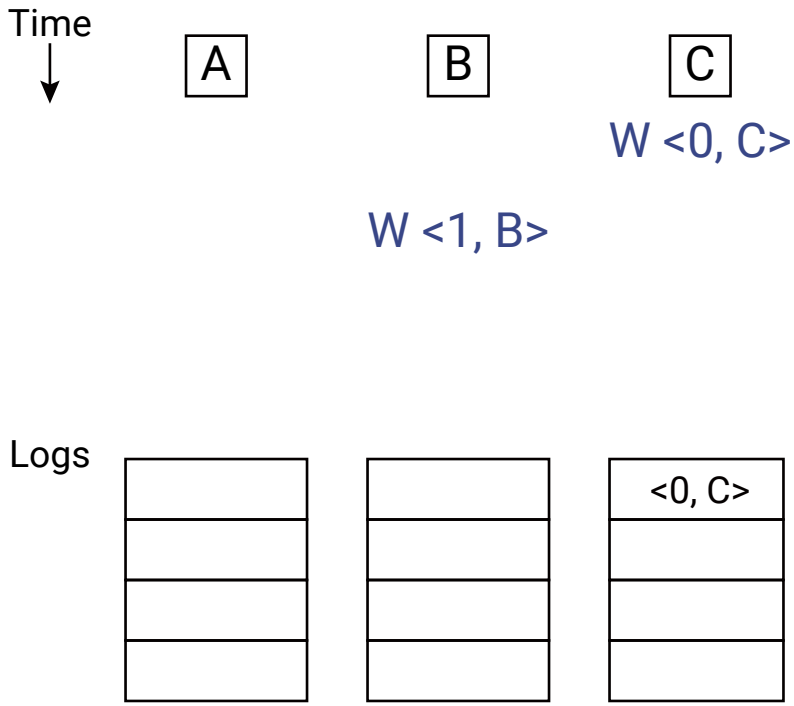
- `⟨local TS, node-id⟩`

Example: Disagreement on Tentative Writes



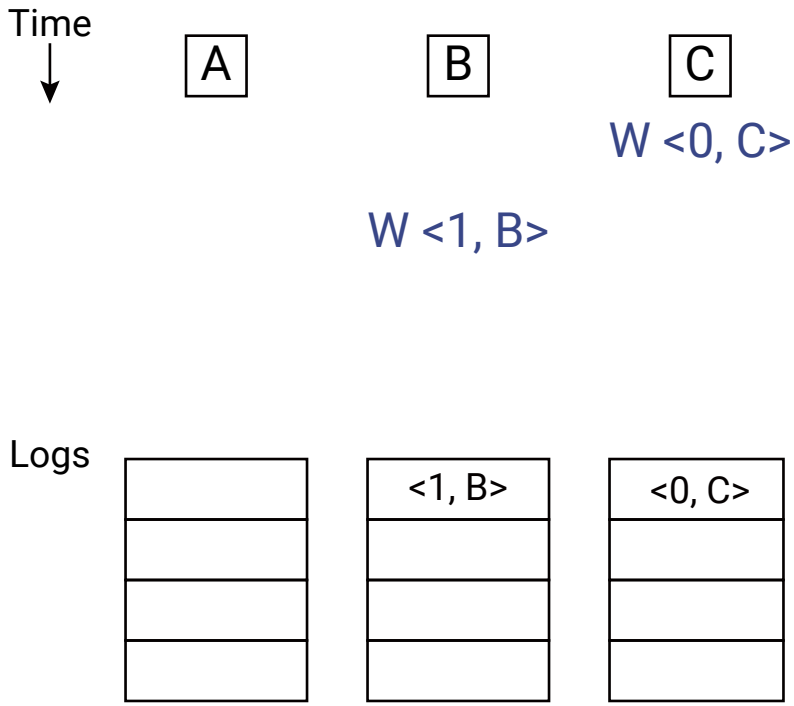
- ⟨local TS, node-id⟩

Example: Disagreement on Tentative Writes



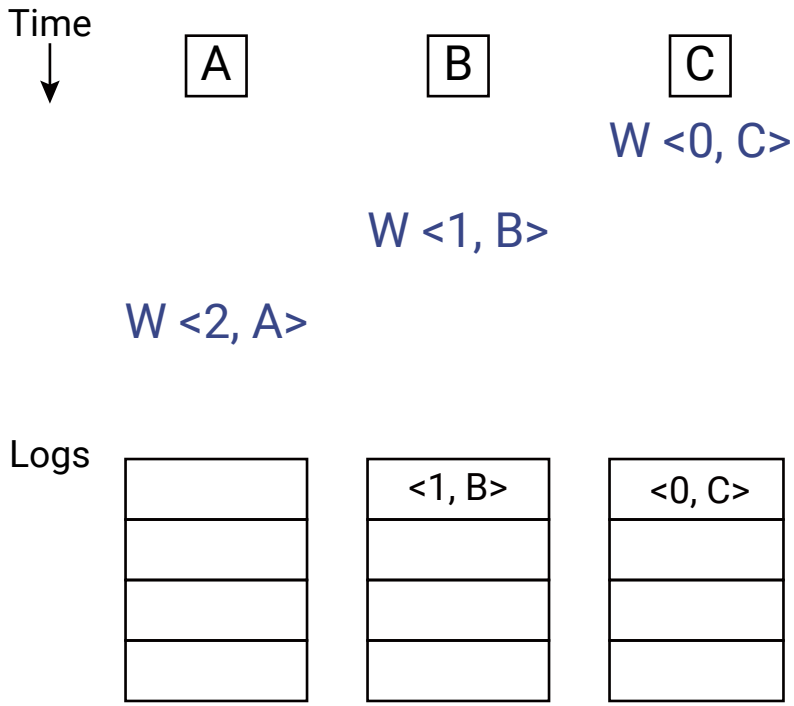
- `〈local TS, node-id〉`

Example: Disagreement on Tentative Writes



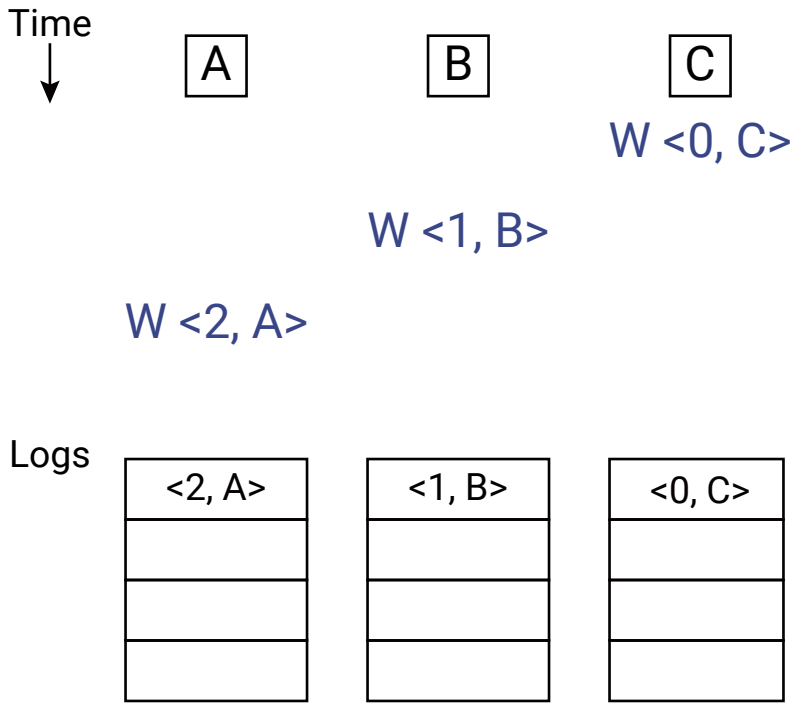
- 〈local TS, node-id〉

Example: Disagreement on Tentative Writes



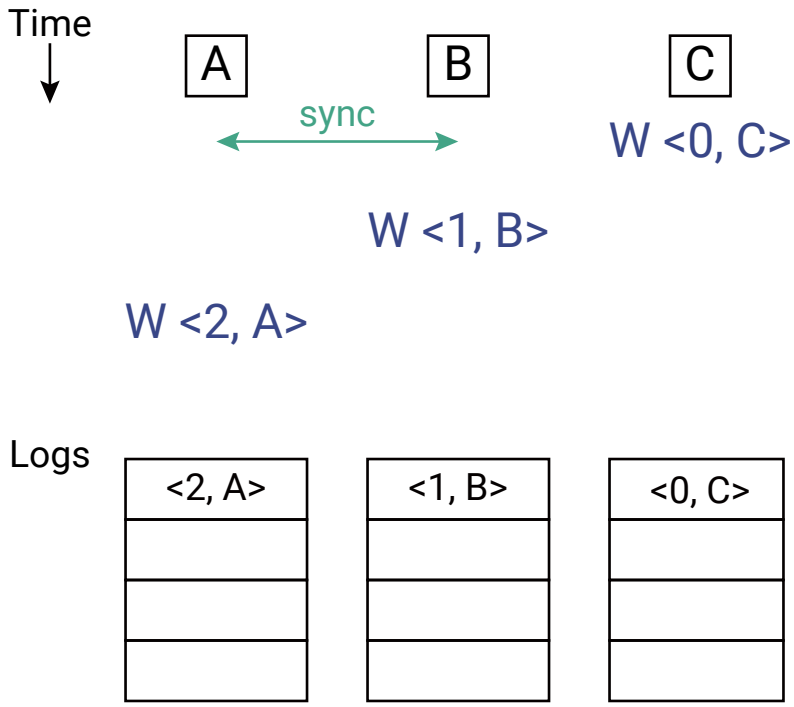
- ⟨local TS, node-id⟩

Example: Disagreement on Tentative Writes



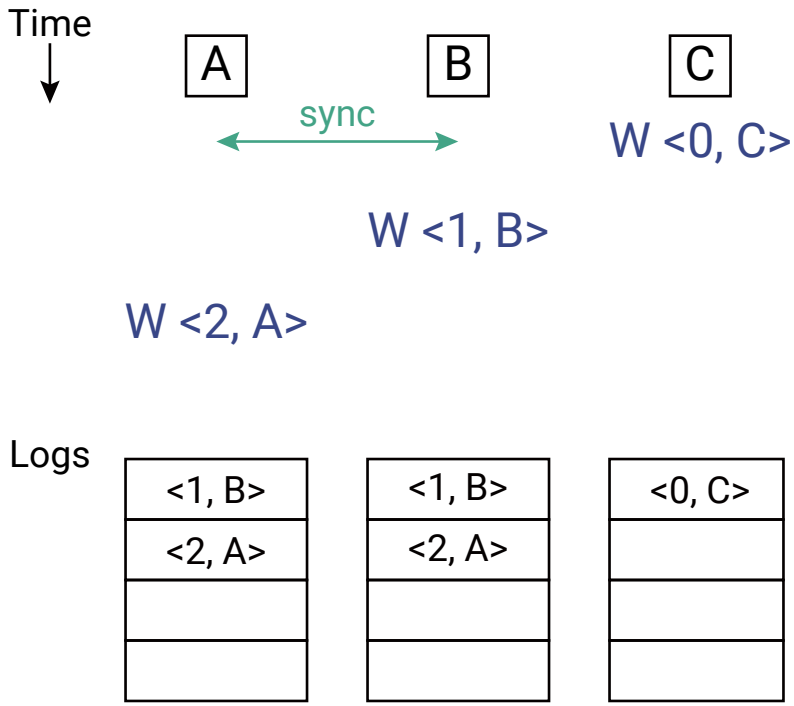
- ⟨local TS, node-id⟩

Example: Disagreement on Tentative Writes



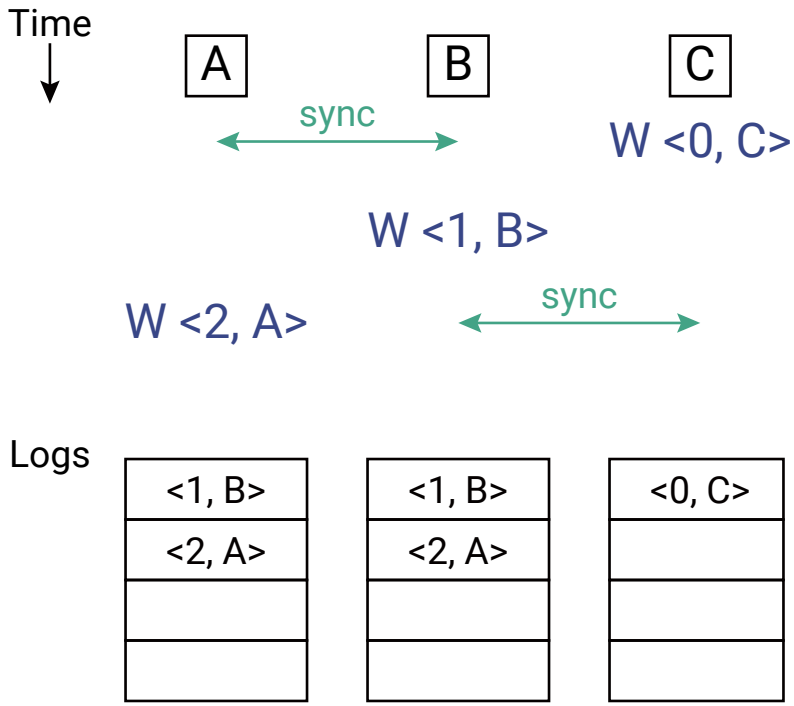
- ⟨local TS, node-id⟩

Example: Disagreement on Tentative Writes



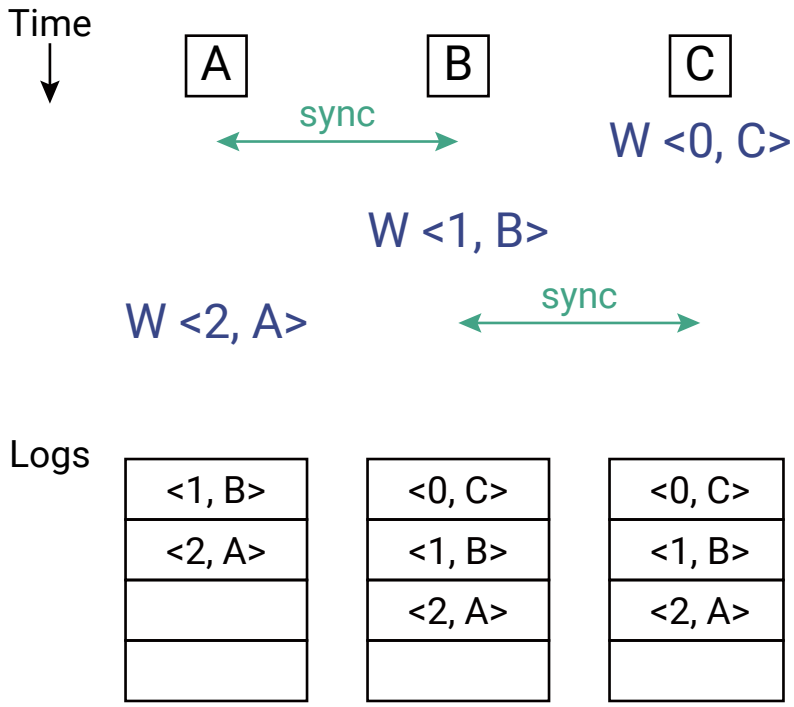
- 〈local TS, node-id〉

Example: Disagreement on Tentative Writes



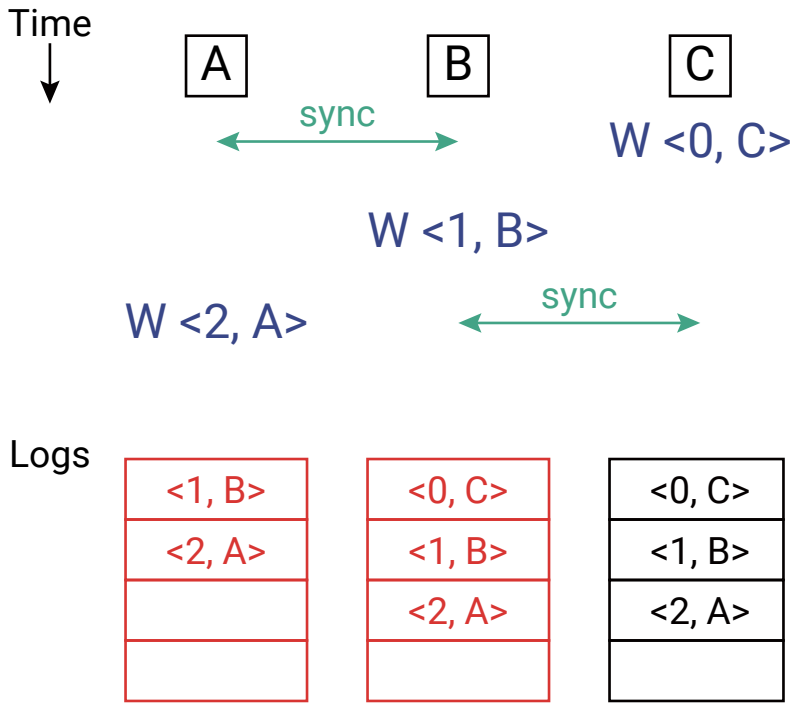
- ⟨local TS, node-id⟩

Example: Disagreement on Tentative Writes



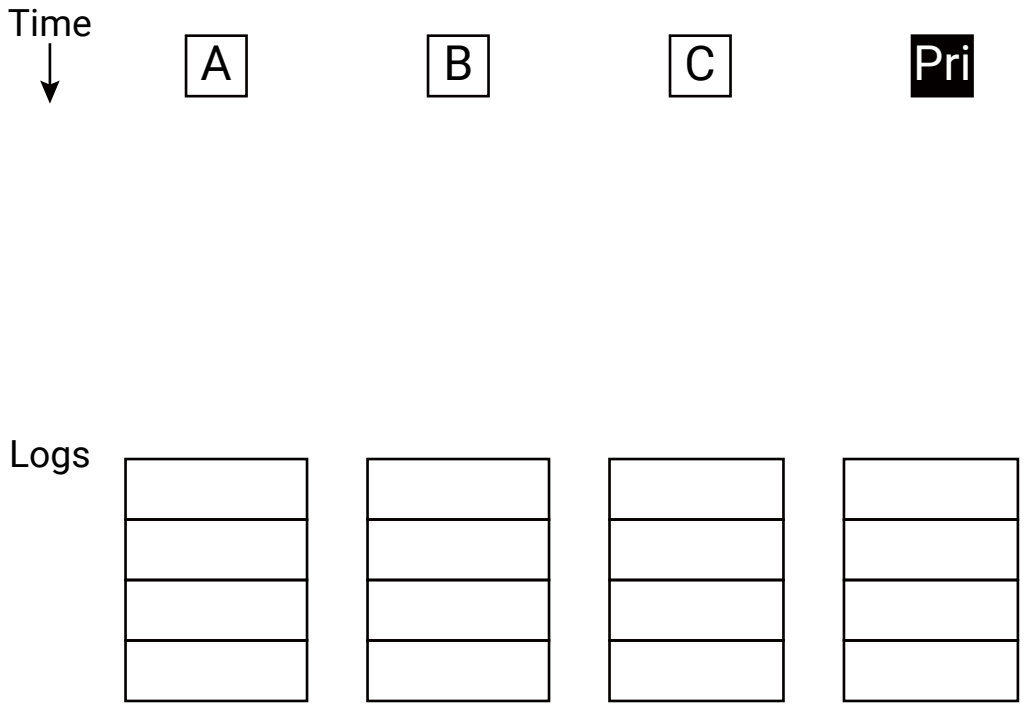
- ⟨local TS, node-id⟩

Example: Disagreement on Tentative Writes



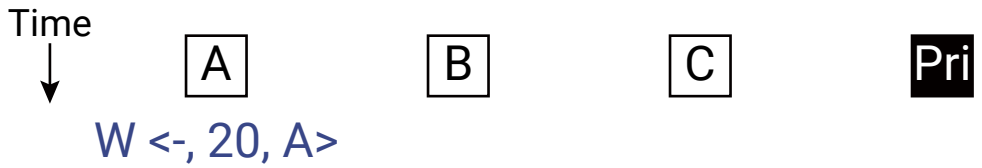
- ⟨local TS, node-id⟩

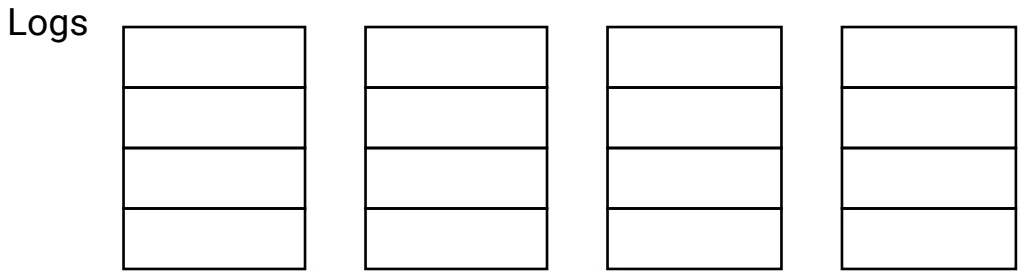
Tentative order ≠ commit order



- 〈CSN, local TS, node-id〉

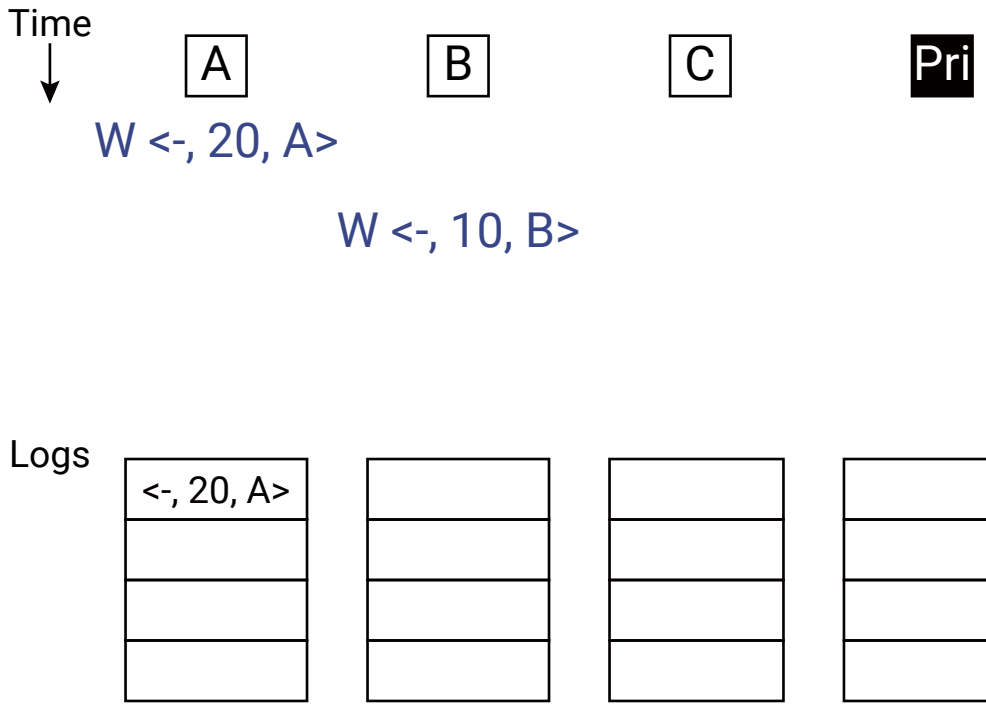
Tentative order ≠ commit order





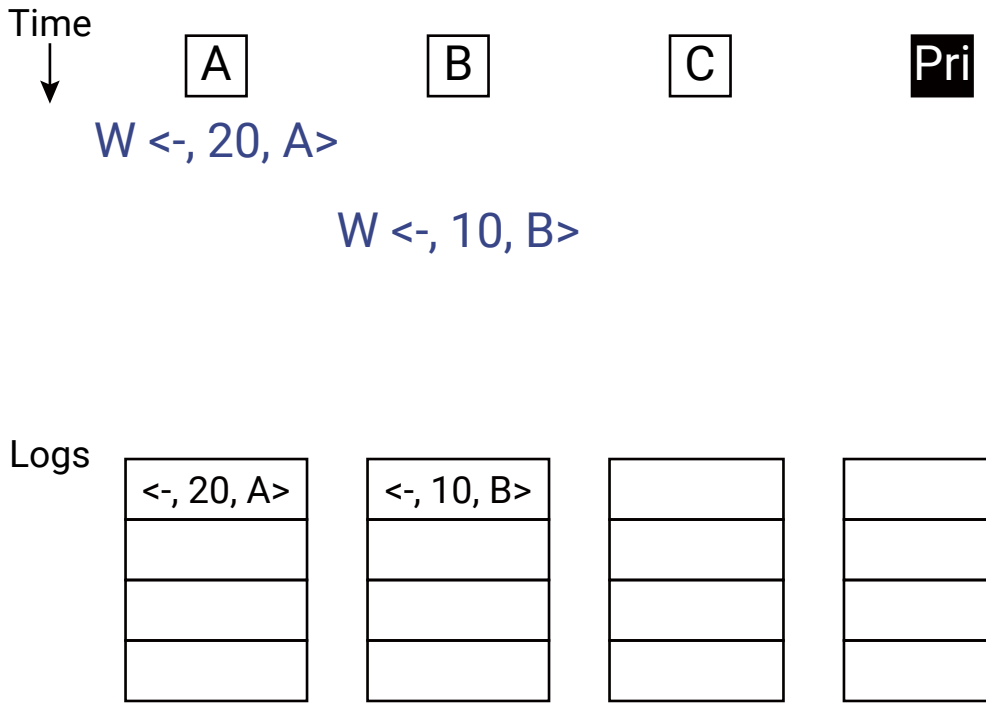
- ⟨CSN, local TS, node-id⟩

Tentative order ≠ commit order



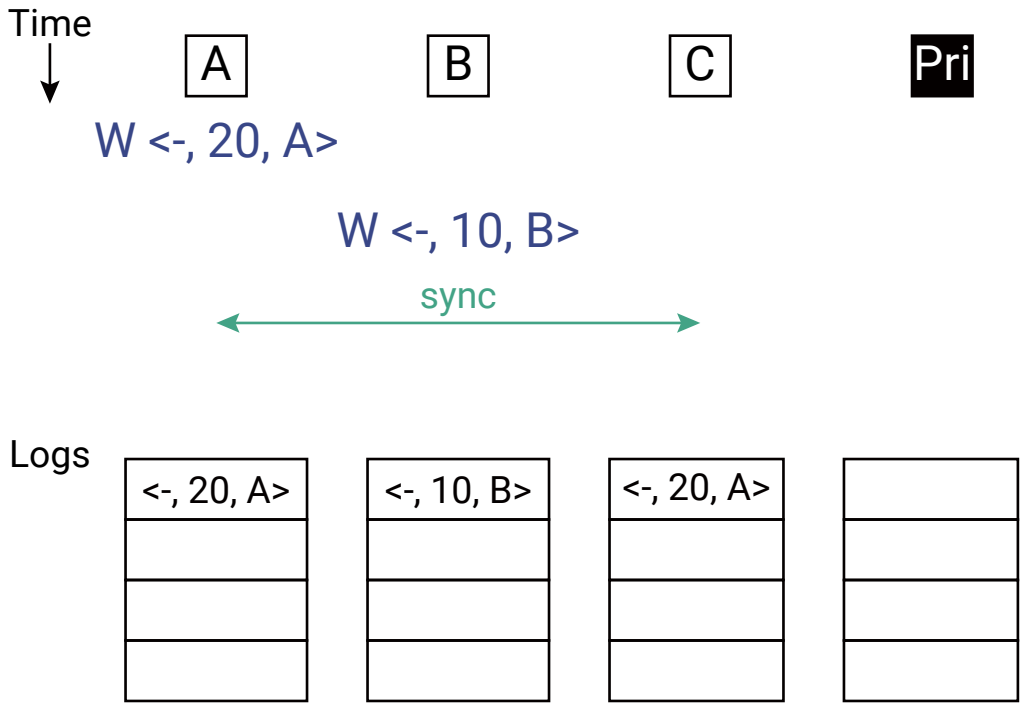
- 〈CSN, local TS, node-id〉

Tentative order ≠ commit order



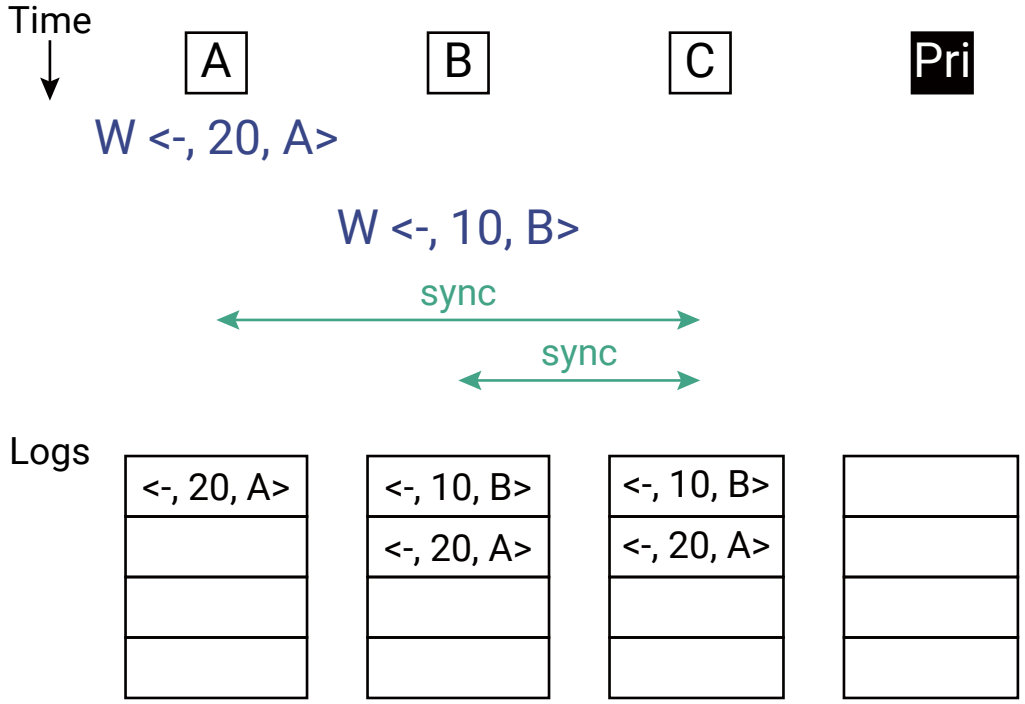
- 〈CSN, local TS, node-id〉

Tentative order ≠ commit order



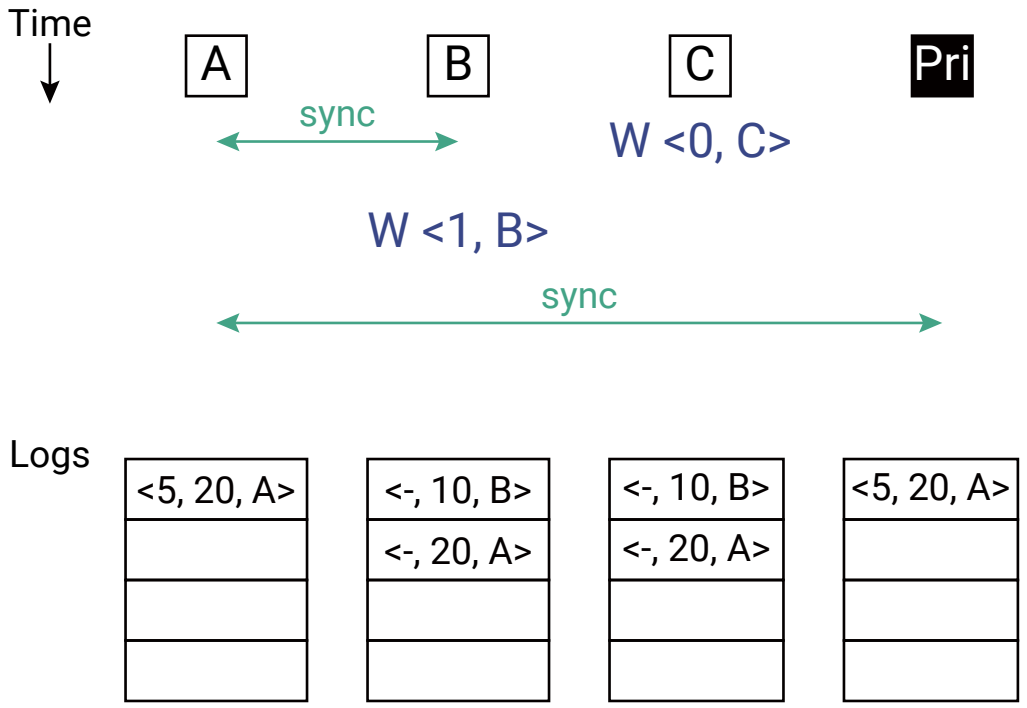
- 〈CSN, local TS, node-id〉

Tentative order ≠ commit order



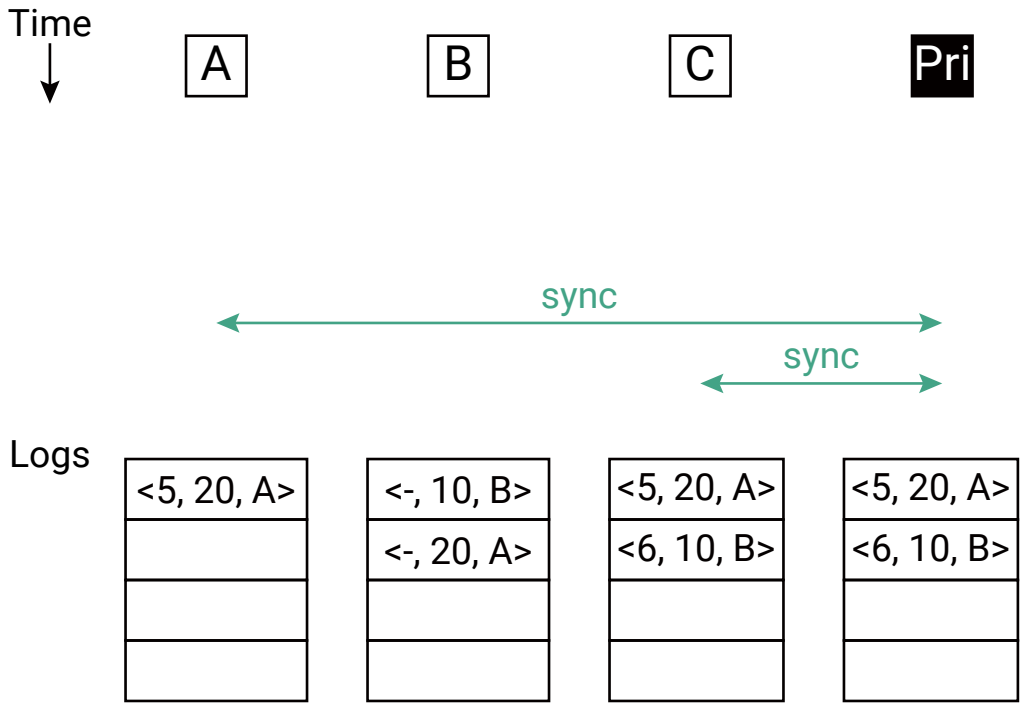
- 〈CSN, local TS, node-id〉

Tentative order ≠ commit order



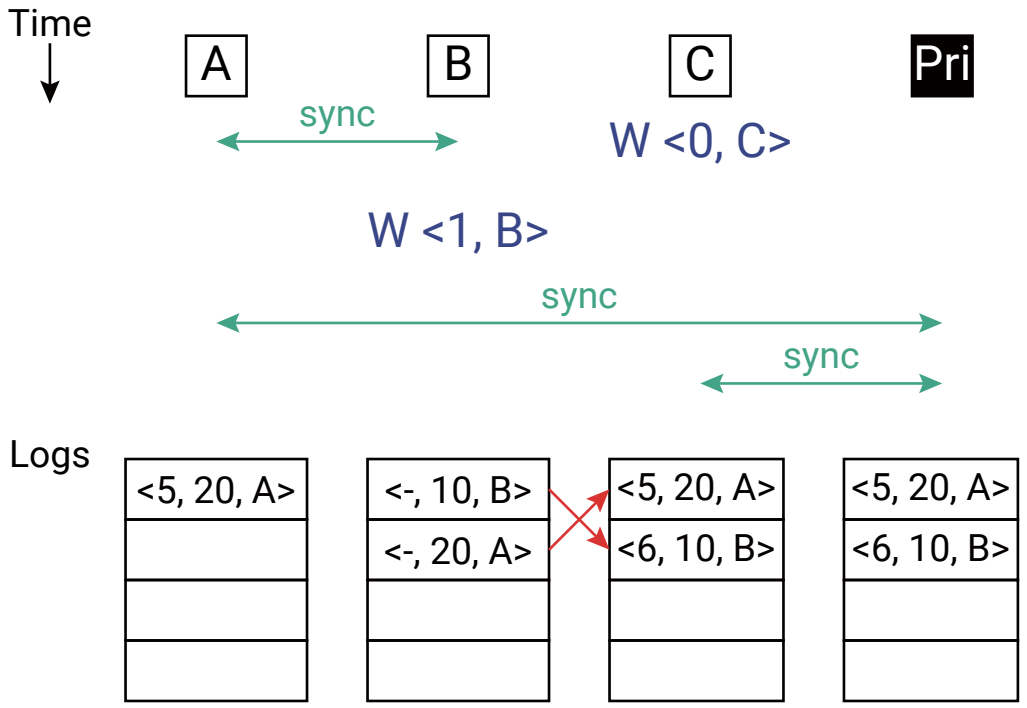
- ⟨CSN, local TS, node-id⟩

Tentative order ≠ commit order



- 〈CSN, local TS, node-id〉

Tentative order ≠ commit order



- ⟨CSN, local TS, node-id⟩

Primary Commit Order Constraint

- Suppose user **creates meeting**, then deletes or changes it
  - What CSN order must these operations have?

Primary Commit Order Constraint

- Suppose user **creates meeting**, then deletes or changes it
  - What CSN order must these operations have?
    - Create first, then delete or modify
    - Must be true in every node's view of tentative log entries, too

Primary Commit Order Constraint

- Suppose user **creates meeting**, then deletes or changes it
  - What CSN order must these operations have?
    - Create first, then delete or modify
    - Must be true in every node's view of tentative log entries, too

Rule Primary's total write order must preserve causal order of writes.

Primary Preserves Causal Order

Rule Primary's total write order must preserve causal order of writes.

- How?

Primary Preserves Causal Order

Rule Primary's total write order must preserve causal order of writes.

- How?
  - Nodes sync **full** logs

Primary Preserves Causal Order

Rule Primary's total write order must preserve causal order of writes.

- How?
  - Nodes sync **full** logs
    - If A $\rightarrow$ B then A must appear in every log before B
  - Primary orders newly synced writes in **tentative order**
    - Primary will commit A and then commit B

How Do We Sync Efficiently?

Goal Two nodes meet and want to ensure they both have the same set of writes

- The Naive Way: Send the entire Write Log to the other node
  - Problem: The log grows forever. Using 10MB of bandwidth to sync 1KB of new data is unacceptable
- The Bayou Way: Anti-Entropy Protocol
  - Nodes only exchange the missing prefix of the log
  - To do this, they need a way to summarize what they already know

Summarizing Knowledge

- Each node maintains a **Version Vector** (also called a Knowledge Vector).
  - A set of pairs: `(Server ID, Max TS)`
  - Example: Node A's vector is `[(A, 100), (B, 50), (C, 20)]`
    - "I have seen all writes from A up to time 100, all from B up to 50, and all from C up to 20."

Rule If Node A has a vector entry `(B, 50)`, it is guaranteed to have every write B ever produced before timestamp 50. (No gaps allowed in the log!

The Anti-Entropy “Handshake”

1. **Contact**: Node A connects to Node B.
2. **Exchange**: Node A sends its Version Vector to Node B.
3. **Identify**: Node B compares A’s vector to its own log.
   - If B has a write from Server C with timestamp 21, and A’s vector says (C, 20), B knows A is missing that write.
4. **Send**: Node B sends only the specific writes that A is missing.
5. **Update**: Node A adds the writes to its log and updates its Version Vector.

Syncing the Commitment Boundary

- Recall: The Primary Node decides when a write is “Committed”
- When nodes sync, they don’t just sync writes; they also sync the Commit Sequence Number (CSN)
- If Node B knows that Write #45 is now “Committed” but Node A still thinks it is “Tentative,” the Anti-Entropy process will update Node A’s status

Result Information about what is “final” spreads through the network like a virus (Epidemic Protocol)

Who Talks to Whom?

- Bayou doesn't force a specific topology. Replicas can sync in any pattern:
  - Direct: Laptop to Server
  - Peer-to-Peer: Laptop to PDA (via Infrared/Bluetooth)
  - Gossip: Randomly picking a neighbor to sync with
- **Anti-Entropy is reconciliation**: It doesn't matter who you talk to; as long as the graph of connections is "eventually" connected, every write will eventually reach every node

Trimming the Log

- When nodes receive new CSNs, can discard all committed log entries seen up to that point
  - Sync protocol $\rightarrow$ CSNs received in order
- Keep copy of whole database as of highest CSN

Result No need to keep years of log data

Let's Step Back

- Is eventual consistency a useful idea?
  - Yes: we want fast writes to local copies (iPhone sync, OneDrive, …)
- Are update conflicts a real problem?
  - Yes: all systems have some more or less awkward solution

Is Bayou's Complexity Warranted?

- Update functions, tentative operations, …
- Only critical if you want peer-to-peer sync
  - i.e., disconnected operation AND ad-hoc connectivity

What Are Bayou's Take-Away Ideas?

1. **Eventual consistency**: if updates stop, all replicas eventually the same
2. **Update functions** for automatic app-driven conflict resolution
3. **Ordered update log** is the real truth, not the database
4. Use **Lamport clocks**: eventual consistency that respects causality

Is Eventual Consistency Enough?

Problem Even if the system eventually agrees, the intermediate "chaos" can break applications or confuse users

- Example:
  1. You post a comment on a photo (Write W1).
  2. You refresh the page.
  3. Your refresh goes to a replica that hasn't received W1 yet.
  4. Result: Your comment "disappears."

Solution We need "Client-Centric" consistency. The user should see a consistent history, even if the whole world doesn't yet

Defining a “Session”

- A Session is an abstraction for a sequence of read and write operations performed by a single user/client.
- The **Strategy**: As a user moves from one replica to another (e.g., switching from Wi-Fi to cellular), the system must ensure the new replica is “at least as up-to-date” as the previous one.
- Bayou provides 4 **Session Guarantees**.

1. Read Your Writes

Definition If a client performs a write W, any subsequent read R by that same client must see the effects of W

- Why it **matters**:
  - You don't want to submit a form, refresh the page, and see an empty form
- **Implementation**: The client keeps a version vector of its own writes and refuses to talk to any replica that hasn't processed those writes yet

2. Monotonic Reads

Definition If a client reads the value of a data item X, any successive reads of X by that client will always return that same value or a more recent value

- **The "Time Travel" Problem**: Without this, you might see an email in your inbox at 10:00 AM, but after refreshing at 10:01 AM, the email is gone because you hit a "staler" replica

Result Data never moves "backward" in time from the user's perspective

3. Writes Follow Reads

Definition A write W by a client that follows a previous read R by the same client is guaranteed to take place on a state that includes the effects of the write that produced R

- Example (Newsgroup/Forum):
  1. You read a post (Post A)
  2. You write a reply (Post B)
  3. **Guarantee**: Anyone who sees your reply (Post B) is guaranteed to also see the original post (Post A)
- It preserves the **causal context** of your actions

4. Monotonic Writes

Definition A write by a client must be processed by any replica before any subsequent write by the same client

- Example (Software Update):
  1. You upload “Version 1” of a file
  2. You upload “Version 2” (which overwrites V1)
  3. **Guarantee**: No replica will ever apply V2 and then later apply V1
- This ensures that your personal sequence of updates is applied in the order you intended

Implementing Guarantees via Session Vectors

- **Client**: The client library maintains two vectors:
  1. Read Vector: The highest version numbers the client has seen
  2. Write Vector: The highest version numbers of the client's own writes
- **Server**: When a client sends a request to a replica, it attaches these vectors
- The **Check**: If the replica's own log is "behind" the client's vectors, the replica must:
  1. Sync with other nodes to catch up, OR
  2. Tell the client "I'm too stale, try another replica"

Summary: The Bayou Blueprint

- **Replicas**: Every node is a full server; no central bottleneck
- **Writes**: Include Dependency Checks and Merge Procedures
- **Ordering**: Lamport Clocks provide a “Tentative” total order
- **Stability**: A “Primary” node assigns “Commit Sequence Numbers” (CSNs)
- **Propagation**: Anti-entropy (Gossip) ensures everyone eventually sees every write
- **Experience**: Session Guarantees keep the world making sense for the user

Copyright